Python

Czwarty wykład

dr inż. Marcin Ciura, Uniwersytet Komisji Edukacji Narodowej

Przepraszam was za to, że nie powiem wam, ile sztuczek dziś wam pokażę :-)

Dam wam wskazówkę:
osiem to też kilka :-)

Co tu robi ta szynka?



Dobry wykład powinien zacząć się od anegdoty

Kiedy programiści rozmawiają o programowaniu, czasem chcą podać przykład składni języka. Zamiast wymyślać od nowa nazwy funkcji i zmiennych, używają zwyczajowych nazw foo i bar, na przykład:

```
def foo():
    return bar
```

Dobry wykład powinien zacząć się od anegdoty

Programiści Pythona też rozmawiają o programowaniu, i też czasem chcą podać przykład składni Pythona. Ale programiści Pythona nie rozmawiają o foo i bar, tylko o mielonce i szynce, na przykład:

```
def spam():
    return ham
```

Jak do tego doszło?

Jak do tego doszło?

Pamiętajcie proszę, że nazwa Python wzięła się od komedii Monty Pythona

Jeśli chcesz obejrzeć skecz o mielonce, polecam ci wpisać w YouTube [monty python mielonka]. Polecam ci wersję live. Jest lepsza niż oryginał :-)

Ale wróćmy do tematu

Proszę robić notatki :-)

Plan na dziś

+ Kilka sztuczek
+ Krotki
+ Funkcje wbudowane:
  - range
  - enumerate
  - sorted
+ Zbiory
+ Importowanie modułów
+ Wybrane funkcje czterech modułów:
  - math
  - statistics
  - random
  - urllib

Sztuczka:
Łączenie operatorów porównania

Łączenie operatorów porównania

Zamiast pisać tak:

```
if x >= 90 and x <= 110:
    print('Około 100')
```

możemy pisać tak:

```
if 90 <= x <= 110:
    print('Około 100')
```

Sztuczka:
Metoda .get słowników

Metoda .get słowników

Jeśli wyraz_pl nie należy do słownika słownik_pl_cz, to wykonanie tej linii programu spowoduje wyjątek, czyli przerwie działanie programu:

```
wyraz_cz = słownik_pl_cz[wyraz_pl]
```

Metoda .get słowników

Aby Python niespodziewanie nie przerwał programu, lepiej pisać tak:

```
if wyraz_pl in słownik_pl_cz:
    wyraz_cz = słownik_pl_cz[wyraz_pl]
else:
    wyraz_cz = '???'
```

Metoda .get słowników

Dzięki metodzie .get możemy to zapisać krócej:

```
wyraz_cz = słownik_pl_cz.get(wyraz_pl, '???')
```

Metoda .get ma dwa argumenty:
+ klucz, którego wartość chcemy pobrać
+ wartość domyślną, którą zwróci ta metoda, jeśli klucza nie ma w słowniku

Metoda .get słowników — zagadka

Co robi taki fragment programu?

```
wyraz_cz = słownik_pl_cz.get(wyraz_pl, wyraz_pl)
```

Lukier składniowy

Na takie sztuczki, które nie zmieniają istoty języka programowania, tylko dodają do niego wygodniejszy sposób zapisu, mówi się „lukier składniowy"

Jeśli ciastko bez lukru jest smaczne, to z lukrem będzie smaczniejsze. Byle nie przedobrzyć :-)

Jeśli język programowania jest wygodny, to z lukrem składniowym będzie wygodniejszy. Byle nie przedobrzyć

Krotki

Krotki

Nazwa „krotka” (“tuple”) pochodzi od wyrazu „krotność”:

+ dwukrotny (double)
+ trzykrotny (triple)
+ czterokrotny (quadruple)
+ pięciokrotny (quintuple)
+ sześciokrotny (sextuple)
+ siedmiokrotny (septuple)

Krotki

Krotkę można by pomylić z listą, tylko że:
+ listy otacza się nawiasami kwadratowymi, a krotki nawiasami okrągłymi ( )
+ elementy list można zmieniać, a krotki są niezmienne
+ elementy jednej listy zazwyczaj są tego samego typu, a elementy jednej krotki niekoniecznie

Krotki

Oto przykłady krotek:

współrzędne = (4, 3)
student = ('Jan', 'Nowak', 'bioinformatyka', 123456)

Krotki

Co robi ten kod?

```
print(współrzędne[0]**2, współrzędne[1]**2)
print(student[1], student[0], student[2])
student[2] = 'biologia'
```

Krotki

Co robi ten kod?

```
print(współrzędne[0]**2, współrzędne[1]**2)
print(student[1], student[0], student[2])
student[2] = 'biologia'
```

16 9

Nowak Jan bioinformatyka

TypeError: 'tuple' object does not support item assignment

Porównywanie krotek

Tak porównujemy krotki. Element po elemencie, od pierwszego elementu:

```
student = ('Kowalski', 'Jan', 123456)
print(student == ('Kowalski', 'Jan', 123456))
print(student == ('Kowalski', 'Jan', 654321))
print(student < ('Nowak', 'Jan', 111111))
True
False
True
```

Krotki

Łańcuchy, liczby i krotki mogą być kluczami słowników, bo są niezmienne

Takie słowniki, które miałyby zmienne klucze, byłyby okropne: program mógłby wstawić do słownika wartość związaną z jakimś kluczem, a potem zmienić ten klucz i zgubić dostęp do wstawionej wartości

Sztuczka:
Przypisanie krotki do krotki

Przypisanie krotek do krotek

Przepraszam, oszukałem was: nawiasy wokół krotek zwykle nie są potrzebne
Po prawej stronie operatora przypisania = zwykle się je pisze. Po lewej stronie operatora przypisania = zwykle się ich nie pisze

```
student = ('Jan', 'Kowalski', 'bioinformatyka', 123456)
imię, nazwisko, kierunek, numer_indeksu = student
```

Przypisanie krotek do krotek

Zamiast pisać tak:

```
if a > b:
    tmp = a  # tmp to skrót od "temporary variable", czyli „tymczasowa zmienna"
    a = b
    b = tmp  # Teraz na pewno a <= b
```

możemy wygodnie zamieniać wartości zmiennych:

```
if a > b:
    a, b = b, a  # Teraz na pewno a <= b
```

Postęp wykładu

Przedstawiłem wam:
- łączenie operatorów porównania
- metodę .get słowników
- przypisanie krotki do krotki

+ Krotki

Teraz omówię:
- złożone operatory przypisania
- domyślne argumenty funkcji
- listy składane
- słowniki składane
- zbiory składane
- prawdziwe i nieprawdziwe obiekty

+ Funkcje wbudowane:
  - **range**
  - **enumerate**
  - **sorted**

+ Zbiory

+ Importowanie modułów
+ Wybrane funkcje czterech modułów:
  - math
  - statistics
  - random
  - urllib

Proszę mi zadać pytanie :-)

Sztuczka:
Złożone operatory przypisania

Złożone operatory przypisania

Zamiast pisać tak:

```
x = x + 5
lista[i] = 2 * lista[i]
słownik[k] = słownik[k] - 1
```

możemy pisać zgodnie z zasadą DRY (Don't Repeat Yourself)

```
x += 5
lista[i] *= 2
słownik[k] -= 1
```

Złożone operatory przypisania

To są złożone operatory przypisania:

+= -= *= /= %= **=

Operator reszty z dzielenia %

Operator % oblicza resztę z dzielenia lewej strony przez prawą:

```
print(10 % 3)
1
print(12 % 4)
0

if x % 2 == 0:
    print(f'{x} to liczba parzysta')
```

Funkcje wbudowane
range, enumerate, sorted

Jest sporo funkcji wbudowanych w Pythona. Omówię 3

Funkcja wbudowana range

Funkcja wbudowana range

Funkcja range generuje ciągi liczb całkowitych
Funkcja range jest często używana w pętlach **for**

Tak możemy zapisywać funkcję range:

range(stop)
range(start, stop)
range(start, stop, krok)

Funkcja wbudowana range — wersja z jednym argumentem

range(stop) generuje kolejne liczby całkowite od 0 do stop-1
Funkcja range nigdy nie generuje wartości stop

```
for i in range(5):
    print(i)
```

```
0
1
2
3
4
```

Funkcja wbudowana range — wersja z dwoma argumentami

range(start, stop) generuje liczby całkowite od start do stop-1
Funkcja range nigdy nie generuje wartości stop

```
for i in range(3, 7):
    print(i)
```

```
3
4
5
6
```

Funkcja wbudowana range — wersja z trzema argumentami

range(start, stop, krok) pozwala określić krok, czyli wartość, o którą zwiększa się kolejna liczba ciągu. Funkcja range nigdy nie generuje wartości stop

```
for i in range(1, 10, 2):
    print(i)
```

```
1
3
5
7
9
```

Funkcja wbudowana range — wersja z trzema argumentami

range(start, stop, krok) pozwala określić krok, czyli wartość, o którą zwiększa się kolejna liczba ciągu. Krok może być ujemny. Funkcja range nigdy nie generuje wartości stop

```
for i in range(5, 0, -1):
    print(i)
```

```
5
4
3
2
1
```

Funkcja wbudowana **enumerate**

Funkcja wbudowana enumerate

Funkcja enumerate generuje krotki (indeks, wartość)

```
imiona = ['Anna', 'Bartek', 'Celina']
for i, imię in enumerate(imiona):
    print(i, imię)
```

```
0 Anna
1 Bartek
2 Celina
```

Funkcja wbudowana enumerate

Argumentem funkcji enumerate może być dowolny obiekt, po którym można iterować: lista, krotka, łańcuch znaków, słownik lub zbiór

```
for i, znak in enumerate('Cześć'):
    print(i, znak)
```

```
0 C
1 z
2 e
3 ś
4 ć
```

Funkcja wbudowana sorted

Funkcja wbudowana sorted

Funkcja sorted sortuje
Funkcja sorted nie zmienia tej kolekcji, która jest jej argumentem, tylko zwraca nową, posortowaną listę

```
lista = sorted(kolekcja)
```

Funkcja wbudowana **sorted**

```
owoce = ['jabłko', 'gruszka', 'banan']  # Lista łańcuchów
print(sorted(owoce))
['banan', 'gruszka', 'jabłko']

liczby = [3, 2, 6, 4]  # Lista liczb
print(sorted(liczby))
[2, 3, 4, 6]
```

Funkcja wbudowana **sorted**

Za pomocą funkcji sorted możemy też sortować inne kolekcje, czyli krotki, zbiory, słowniki (według kluczy)

Funkcja wbudowana **sorted**

Funkcja sorted z dodatkowym argumentem reverse=True sortuje kolekcje w kolejności malejącej

```
liczby = [3, 2, 6, 4]
print(sorted(liczby, reverse=True))
[6, 4, 3, 2]
```

Proszę mi zadać pytanie :-)

Sztuczka:
Domyślne argumenty funkcji

Funkcja print — uzupełnienie

Kiedy funkcja print wypisze swój ostatni argument, domyślnie przechodzi do nowej linii

A kiedy podamy do funkcji print argument end, funkcja print wypisze ten argument zamiast przechodzić do nowej linii

Funkcja print z argumentem end — przykład

```
print('Witaj,')
print('świecie :-)')
```

```
Witaj,
świecie :-)
```

```
print('Witaj,', end=' ')
print('świecie :-)')
```

```
Witaj, świecie :-)
```

Funkcja print — uzupełnienie

Argument end funkcji print ma domyślną wartość '\n'

Argument reverse funkcji sorted ma domyślną wartość False

Wartość domyślna to taka wartość, którą przyjmuje argument, gdy wywołujemy funkcję bez tego argumentu

Funkcja print z argumentem end — przykład

Proszę cię, przeczytaj ten kod dokładnie:

```
print('Liczby: ', end='')
for i in range(1, 4):
    print(i, end=' ')
print('Gotowe!')
```

```
Liczby: 1 2 3 Gotowe!
```

Domyślne argumenty funkcji

My też możemy definiować funkcje z argumentami domyślnymi. Wtedy w nagłówku funkcji przypisujemy tym argumentom wartości za pomocą operatora =

Argumenty z wartościami domyślnymi muszą być na końcu listy argumentów

Domyślne argumenty funkcji

```
def powitaj(imię, wiadomość='Witaj'):
    """Wypisuje wiadomość i imię."""
    print(f'{wiadomość}, {imię}!')

powitaj('Jan')
Witaj, Jan!
powitaj('Anna', 'Cześć')
Cześć, Anna!
```

Zbiory

Zbiory

Zbiory to takie skromniejsze słowniki
Zbiory mają tylko klucze, nie mają wartości
Każdy klucz w zbiorze występuje jeden raz
Kolejność elementów w zbiorze nie ma znaczenia

```
owoce = {'jabłko', 'gruszka', 'banan'}
liczby_pierwsze = {2, 3, 5, 7, 11, 13, 17, 19}
```

Uwaga: {} to pusty słownik, a nie pusty zbiór. Zbiór pusty tworzymy tak:

```
zbiór_pusty = set()
```

Działania na zbiorach

Czy element należy do zbioru:

```
if el in zb:
    print('zmienna el należy do zbioru zb')

if el not in zb:
    print('zmienna el nie należy do zbioru zb')
```

Metoda .add

Tak dodajemy 1 element do zbioru:

```
owoce = {'jabłko', 'gruszka', 'banan'}
owoce.add('pomarańcza')
```

Zasada DRY

Proszę zwrócić uwagę na następny slajd
Dzięki zbiorom nie powtarzamy na tym slajdzie
6 razy fragmentu litery ==

To przykład zastosowania zasady

DRY

Po co nam są zbiory

Zamiast pisać tak:

```
if (litery == 'cz' or litery ==
    'dz' or litery == 'dź' or
    litery == 'dż' or litery ==
    'rz' or litery == 'sz'):
    print(f'{litery} to dwuznak')
```

możemy pisać tak:

```
dwuznaki = {
    'cz', 'dz', 'dź', 'dż', 'rz', 'sz',
}
if litery in dwuznaki:
    print(f'{litery} to dwuznak')
```

Działania na zbiorach

Suma dwóch zbiorów (a | b) zawiera te elementy, które należą co najmniej do jednego z tych zbiorów:

```
małe_liczby = {1, 2, 3, 4}
liczby_pierwsze = {2, 3, 5, 7, 11}
suma_zbiorów = małe_liczby | liczby_pierwsze
print(suma_zbiorów)
{1, 2, 3, 4, 5, 7, 11}
```

Działania na zbiorach

Część wspólna dwóch zbiorów (a & b) zawiera te elementy, które należą do obu tych zbiorów naraz:

```
małe_liczby = {1, 2, 3, 4}
liczby_pierwsze = {2, 3, 5, 7, 11}
część_wspólna_zbiorów = małe_liczby & liczby_pierwsze
print(część_wspólna_zbiorów)
{2, 3}
```

Działania na zbiorach

Różnica dwóch zbiorów (a - b) zawiera te elementy, które należą do pierwszego zbioru i nie należą do drugiego zbioru:

```
małe_liczby = {1, 2, 3, 4}
liczby_pierwsze = {2, 3, 5, 7, 11}
różnica_zbiorów = małe_liczby - liczby_pierwsze
print(różnica_zbiorów)
{1, 4}
```

Zbiory — podsumowanie

```
zbiór = {elem1, elem2}
zbiór_pusty = set()

if el in zbiór:
    print('Należy')
if el not in zbiór:
    print('Nie należy')
```

```
suma_zbiorów = z1 | z2
część_wspólna = z1 & z2
różnica_zbiorów = z1 - z2
```

Jak nazywać zmienne?

Przypomnijmy sobie ten slajd:

```
if el in z:
    print('zmienna el należy do zbioru z')

if el not in z:
    print('zmienna el nie należy do zbioru z')
```

Każdy nazywa zmienne tak, jak chce :-)

Jak nazywać zmienne?

Każdy nazywa zmienne tak, jak chce :-)

Warto pamiętać, że czytelnik programu nie musi znać tych samych skrótów, co my. Dlatego odradzam skracanie nazw

Ale ważny jest kontekst, w którym używamy zmiennej.
Jeśli z kontekstu wynika, co znaczy skrót, skrót jest OK :-)

Sztuczka:
Listy składane

Listy składane

Dzięki liście składanej możemy w jednej linii kodu utworzyć listę na podstawie innych kolekcji.

Tak zapisujemy proste listy składane:

[wyrażenie **for** element **in** kolekcja]

Listy składane

Zamiast pisać tak:

```
sześciany = []
for i in range(10):
    sześciany.append(i**3)
```

możemy pisać tak:

```
sześciany = [i**3 for i in range(10)]
```

Listy składane z warunkiem

Listy składane z warunkiem zapisujemy tak:

[wyrażenie **for** element **in** kolekcja **if** warunek]

Listy składane

Stwórz listę tych wyrazów, które mają więcej niż 5 liter:

```
wyrazy = ['jabłko', 'kiwi', 'banan', 'arbuz', 'ananas']
długie_wyrazy = [s for s in wyrazy if len(s) > 5]

print(długie_wyrazy)
['jabłko', 'ananas']
```

Zbiory składane

Zbiory składane

Oprócz list składanych są też zbiory składane i słowniki składane

Zbiory składane tworzymy tak samo, jak listy składane, tylko otaczamy je klamrami { }

{wyrażenie **for** element **in** kolekcja}
{wyrażenie **for** element **in** kolekcja **if** warunek}

Zbiory składane

Wypisz listę wyrazów bez powtórzeń, posortowaną alfabetycznie:

```
tekst = 'Padał deszcz, padał na dach, padał na okna'
print(sorted({wyraz.lower() for wyraz in tekst.split()}))
['dach,', 'deszcz,', 'na', 'okna', 'padał']
```

Słowniki składane

Słowniki składane

Słowniki składane tworzymy podobnie do list składanych i zbiorów składanych

{klucz: wartość **for** element **in** kolekcja}
{klucz: wartość **for** element **in** kolekcja **if** warunek}

Słowniki składane

Stwórz słownik czesko-polski ze słownika polsko-czeskiego:

```
słownik_cz_pl = {cz: pl for pl, cz in słownik_pl_cz.items()}
```

Słowniki składane

Stwórz taki słownik, w którym kluczami są wyrazy,
a wartościami długości tych wyrazów:

```
wyrazy = ['jabłko', 'kiwi', 'banan', 'arbuz', 'ananas']
długości_wyrazów = {w: len(w) for w in wyrazy}
print(długości_wyrazów)
```

```
{'jabłko': 6, 'kiwi': 4, 'banan': 5, 'arbuz': 5, 'ananas': 6}
```

Proszę mi zadać pytanie :-)

Niedługo będzie czas na szynkę



Sztuczka:
Prawdziwe i nieprawdziwe obiekty

Prawdziwe i nieprawdziwe obiekty

Każdy obiekt w Pythonie ma wartość logiczną, której możemy użyć bezpośrednio w instrukcji warunkowej **if**:

```
if obiekt:
    print('Obiekt jest prawdziwy (truthy)')

if not obiekt:
    print('Obiekt jest nieprawdziwy (falsy)')
```

Nieprawdziwe obiekty

False — jedna z dwóch wartości logicznych
0 i 0.0 — zero
'' lub "" — pusty łańcuch znaków
[] — pusta lista
() — pusta krotka
{} — pusty słownik
set() — zbiór pusty
None — specjalny obiekt, który reprezentuje brak wartości

Prawdziwe obiekty

Wszystkie inne obiekty, na przykład:

True — jedna z dwóch wartości logicznych
2.5 — niezerowe liczby
"szynka" — niepuste łańcuchy znaków
['mielonka', 'szynka'] — niepuste listy
(3, 5) — niepuste krotki
{'ser': 4.99, 'szynka': 49.99} — niepuste słowniki
{'Ala', 'As'} — niepuste zbiory

Prawdziwe obiekty

Zamiast tak sprawdzać, czy lista jest pusta, czy nie:

```
if lista != []:
    print('Lista nie jest pusta')
else:
    print('Lista jest pusta')
```

albo zamiast tak sprawdzać:

```
if len(lista) > 0:
    print('Lista nie jest pusta')
else:
    print('Lista jest pusta')
```

można pisać tak, jak na następnym slajdzie

Prawdziwe obiekty

Można pisać tak:

```
if lista:
    print('Lista nie jest pusta')
else:
    print('Lista jest pusta')
```

albo tak*:

```
if not lista:
    print('Lista jest pusta')
else:
    print('Lista nie jest pusta')
```

* Radzę unikać operatora **not** w warunkach, jeśli to możliwe

Importowanie modułów

Importowanie modułów

Takie funkcje, które przydają się w wielu programach, zapisuje się w modułach. Zwykle w jednym module jest wiele funkcji

Moduł to po prostu plik tekstowy z rozszerzeniem .py

Aby używać funkcji z jakiegoś modułu, należy zaimportować ten moduł

Importowanie modułów

Tak się importuje moduł:

import nazwa_modułu

Dobry zwyczaj: importujmy moduły na początku innych modułów

Tak się potem wywołuje funkcję z tego modułu:

nazwa_modułu.nazwa_funkcji(argumenty_funkcji)

Importowanie modułów

Tak można importować moduł, który ma długą nazwę:

```
import długa_nazwa_modułu as dnm
```

Tak się potem wywołuje funkcje z tego modułu:

```
dnm.nazwa_funkcji(argumenty_funkcji)
```

Importowanie modułów

Tak można importować z modułu tylko wybrane funkcje:

from nazwa_modułu **import** funkcja1, funkcja2, funkcja3

Potem wywołujemy te funkcje tak, jakby były zdefiniowane w tym module, w którym są wywoływane
Nie podajemy wtedy nazwy tego modułu, w którym są zdefiniowane:

funkcja1(argumenty_funkcji)

Importowanie modułów

Tak można zaśmiecić „przestrzeń nazw" nie wiadomo czym. Proszę tak nie robić:

from nazwa_modułu **import** *

Biblioteka standardowa

Biblioteka standardowa

Python szuka modułów w określonej kolejności:

+ najpierw sprawdza ten folder, z którego uruchomiono program
+ potem sprawdza foldery ze zmiennej sys.path

Foldery ze zmiennej sys.path zawierają bibliotekę standardową i być może biblioteki zewnętrzne

Niektóre moduły biblioteki standardowej

Biblioteka standardowa jest zawsze zainstalowana razem z Pythonem
Biblioteki zewnętrzne trzeba instalować osobno

W bibliotece standardowej Pythona jest około 160 modułów. „Baterie są w zestawie :-)" Omówię kilka funkcji z 4 modułów

Moduł math

Moduł math

Moduł math zawiera popularne funkcje i stałe matematyczne

```
import math
print(math.pi)
```

```
3.141592653589793
```

Moduł math

Funkcja math.log(x, podstawa=e) zwraca logarytm o danej podstawie z liczby x. Jeśli nie podamy podstawy, ta funkcja używa podstawy e=2.718281828459045…*, czyli zwraca logarytm naturalny

* Liczba e z dokładnością do 15 miejsc po przecinku równa się 2 przecinek 7, potem dwa razy rok urodzenia Lwa Tołstoja, czyli 1828, a potem kąty równoramiennego trójkąta prostokątnego: 45, 90, 45

Moduł math

Funkcje math.sin(x), math.cos(x) i math.tan(x)* zwracają sinus, cosinus i tangens liczby x

Uwaga: liczba x musi być podana w mierze łukowej, czyli w radianach, nie w stopniach

* Niektórzy myślą że język symboli matematycznych jest uniwersalny. To prawda, ale są drobne różnice między narodami. My skracamy tangens do „tg", a Anglosasi do „tan". Ale to jeszcze nic: Włosi nazywają sinus „seno", więc skracają go do „sen" :-)

Ale, ale. Widzę, że niektórym zbiera się na sen :-)
Pytanie na pobudkę:
Ile było sztuczek?

Moduł math

Funkcja math.sqrt (od „square root”) oblicza pierwiastek kwadratowy. Pierwiastek kwadratowy daje ten sam wynik, co podniesienie liczby do potęgi 0.5:

```
import math
print(math.sqrt(9))
print(9 ** 0.5)
3.0
3.0
```

Moduł statistics

Moduł statistics

Moduł statistics zawiera podstawowe funkcje statystyczne
Funkcja statistics.mean oblicza średnią arytmetyczną listy liczb

```
import statistics
ceny = [2, 5, 4, 4]
print(statistics.mean(lista))
print(sum(lista) / len(lista))
3.75
3.75
```

Moduł statistics

Funkcja statistics.median oblicza medianę, czyli liczbę pośrodku listy liczb
Ta lista nie musi być posortowana

```
import statistics
print(statistics.median([1, 2, 4, 8, 3]))
```
```
3
```

Moduł random

Moduł random

Moduł random służy do generowania losowych liczb

Funkcja random.randint(a, b) (od „random integer”) zwraca przy każdym wywołaniu losową liczbę całkowitą między a i b włącznie

Tak się symuluje rzut kostką do gry:

```
import random
print(random.randint(1, 6), random.randint(1, 6), random.randint(1, 6))
```

```
4 5 2
```

Moduł random

Funkcja random.choice(kolekcja) po każdym wywołaniu zwraca losowy element kolekcji. Nie zmienia tej kolekcji. Ta kolekcja nie może być pusta

```
import random
owoce = ['jabłko', 'gruszka', 'banan']
print(random.choice(owoce), random.choice(owoce),
    random.choice(owoce))
banan gruszka banan
```

Moduł urllib

Moduł urllib

Pakiet modułów urllib (od „URL library”; URL to Uniform Resource Locator, czyli adres internetowy) służy do odczytywania danych z Internetu
Najważniejszy moduł tego pakietu to urllib.request

Moduł urllib

Oto szablon pobierania danych z Internetu:

```
import urllib.request
with urllib.request.urlopen('https://www.example.com') as response:
    print(response.read().decode('utf-8'))
```

To wszystko na dziś z wykładu
Proszę mi zadać pytanie :-)

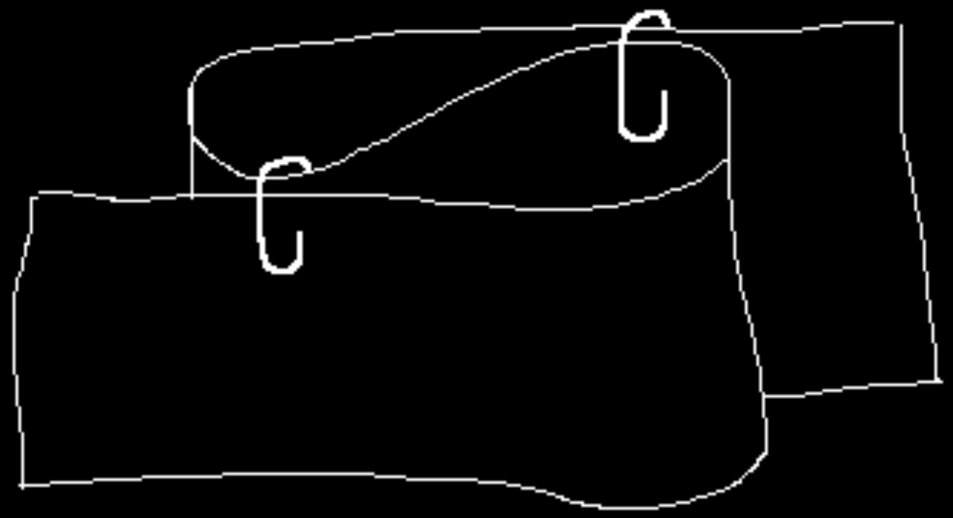
Czy ktoś z was czegokolwiek
nie zrozumiał z tego wykładu?

Ale to nie wszystko
Czas na ostatnią sztuczkę
Kto chce, może wyjść :-)

Jeśli samodzielnie czytasz tę przezentację,
zachęcam cię: zrób tę sztuczkę
Wynik cię zaskoczy :-)

Sztuczka, którą odkrył Bill Bowman z Seattle

1. Zegnij banknot w literę S i zepnij go dwoma spinaczami



2. Śmiało pociągnij za końce banknotu. Nie podrze się :-)

Do zobaczenia na następnym wykładzie
o wyrażeniach regularnych
Gdybym był genetykiem, z całego Pythona najważniejsze
dla mnie byłyby wyrażenia regularne :-)

Źródła zdjęć

Materiały reklamowe o szynce

https://commons.wikimedia.org/wiki/File:30_degree_rotations_expressed_in_radian_measure.svg